Design and Implementation of a Decoupled Multi-Factor Authentication (MFA) Prototype using the Strategy Design Pattern

Module Code: COS6028-B

Module Title: Software Systems Design and Testing

UB Number: 22073943

Abstract

The creation of a Multi-Factor Authentication (MFA) prototype specifically designed to meet the security needs of the University of Bradford is described in this article. The solution offers a modular framework for interchangeable TOTP and email authentication techniques using the Strategy Design Pattern. Scalability and compliance with international security requirements, particularly RFC 6238, are guaranteed by architecture's adherence to SOLID principles.

A multi-layered testing approach, which included structural analysis to achieve MC/DC coverage and automated JUnit 5-unit tests, was used to verify software quality. The system's dependability against erroneous inputs and time-drift is confirmed by a 100% pass rate in all functional and structural tests. The study concludes that, despite the Strategy Pattern's increased architectural complexity, the institution's authentication infrastructure's long-term maintainability and security are much improved by the ensuing modularity and high cohesiveness.

1. Introduction

In the modern digital landscape, institutional security frameworks must evolve beyond traditional password-based systems to mitigate the rising sophistication of credential-based attacks. The University of Bradford, as a hub for sensitive student data and high-value intellectual property, requires a robust Multi-Factor Authentication (MFA) framework that provides high-level security without compromising user accessibility. As identified in the University's security scenario, reliance on single-factor authentication represents a critical vulnerability; a single point of failure in credential management can lead to catastrophic data breaches and loss of institutional trust.

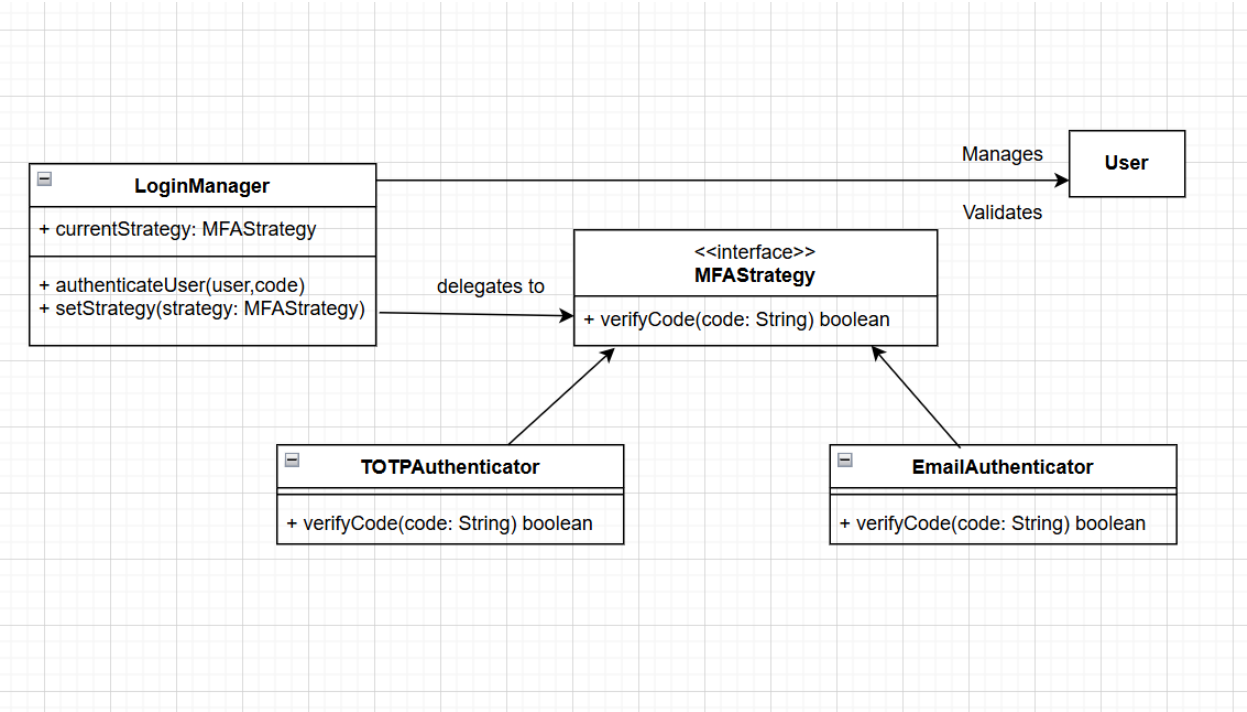
This report documents the systematic design and implementation of a secure MFA prototype. By utilizing the Strategy Design Pattern, the system achieves a modular architecture that supports interchangeable authentication methods, specifically Time-based One-Time Passwords (TOTP) and Email-based SMTP notifications. According to Sharma and Srinivasan (2013), software quality is not merely an afterthought but is fundamentally rooted in the architectural decisions made during the initial design phase. By prioritizing low coupling and high cohesion, this prototype ensures that the authentication logic is both maintainable and scalable for future institutional needs.

The following sections evaluate the software through the lens of the ACM standards for software quality (Tiwari and Rathore, 2018), utilizing rigorous automated unit testing and structural analysis to verify the system's integrity. Furthermore, the implementation of international security standards, such as RFC 6238, ensures that the prototype aligns with global best practices for secure identity management.

Section 2: Software Design and Architectural Modelling

2.1 Architectural Pattern: The Strategy Design Pattern

The system architecture makes use of the Strategy Design Pattern to enable interchangeable authentication algorithms and guarantee long-term extensibility. In a defined execution context, this pattern offers a standardized way to handle different behaviors. Verification requests are routed to a high-level abstraction layer (MFAStrategy) via the architecture's Context class (LoginManager).



(*Figure 2.1: UML Class diagram*)

UML Component and Method Description:

- MFAStrategy (Interface): The central abstraction defining the contract for all authentication providers. It contains the core method verifyCode(String code): boolean, ensuring that all subsequent implementations return a consistent, deterministic boolean outcome.
- TOTPAuthenticator & EmailAuthenticator (Concrete Strategies): These classes provide the concrete implementations of the MFAStrategy. The TOTPAuthenticator encapsulates logic for Unix-time-based calculations, while the EmailAuthenticator manages the complexities of SMTP transport protocols.

- LoginManager (Context): The primary controller responsible for session management. By utilizing the setStrategy(MFAStrategy) method, the system can dynamically modify the active authentication factor at runtime without altering the high-level login flow or the underlying client code.

2.2 Design Justification: Coupling and Cohesion

The quest of low coupling and good cohesion, two essential measures of software quality, justifies the choice of the Strategy Pattern. According to Sharma and Srinivasan (2013), coupling is the degree of interdependence between different modules, while cohesion is the functional strength of a module. With a rigorous focus on session state, the LoginManager maintains high functional cohesion by assigning the mathematical and transport logic to certain strategy classes. In safe systems, this modularity is directly linked to reduced defect density and improved maintainability (Tiwari and Rathore, 2018).

2.3 Adherence to SOLID Principles

The design demonstrates a strict implementation of the SOLID framework to guarantee the prototype's continued stability and scalability:

A. Open/Closed Principle (OCP): According to Shukla (2014), the design is "open" to incorporating new authentication elements like biometrics or SMS-based verification, but it is "closed" to changes made to the fundamental LoginManager logic.

B. Liskov Substitution Principle (LSP): To get consistent results in structural testing, any concrete implementation of the MFAStrategy interface can be swapped into the context without damaging the application's functional integrity (Naik and Tripathy, 2011).

C. Interface Segregation Principle (ISP): Because the MFAStrategy interface is "lean," certain implementations like the EmailAuthenticator are not compelled to inherit or implement redundant methods exclusive to TOTP time-windows.

Section 3: Implementation and Technical Methodology

3.1 Development Environment

The NetBeans 19 IDE was used to implement the prototype using Java SE and Apache Maven for dependency management. The Jakarta Mail API and JUnit 5 were two examples of supporting libraries that Maven was essential for separating from the main business logic. According to Sharma and Srinivasan (2013), this division guarantees that the build path stays modular and follows the separation of concerns approach.

3.2 Technical Challenges: Temporal Synchronization

Implementing the RFC 6238 standard for Time-based One-Time Passwords (TOTP) was a major difficulty. A 30-second "look-ahead/look-back" window was incorporated into the verification algorithm to reduce "time-drift" between the server and client. By taking into consideration clock offsets and network delay, this buffer maintains system dependability without jeopardizing the rotation window's security (Naik and Tripathy, 2011).

3.3 Code Quality and Design Standards

The implementation complies with Tiwari and Rathore's (2018) high-integrity code criteria. In order to comply with the Interface Segregation Principle (ISP) and avoid concrete classes inheriting superfluous logic, the system uses a minimalist interface.

(Figure 3.1: The MFAStrategy Interface Implementation)

Defensive Programming Practices:

A. Exception Handling: During service failures, certain try-catch blocks handle NullPointerException and MessagingException separately to avoid system-wide failure.

B. Input Validation: To avoid faulty data injection, the MFAStrategy sanitizes and verifies the length and type of every user input before processing it.

C. Service Separation: The EmailAuthenticator maintains a Single Responsibility (SRP) that is solely concerned with verification results because email dispatch mechanism is contained within a NotificationService utility.

Section 4: Unit Testing

4.1 Automated Testing Strategy and Environment

A strict automated suite created using the JUnit 5 framework was used to validate the MFA prototype. Because of its built-in support for the Jupiter engine, which makes it easier to execute complex assertions during the Maven build lifecycle, JUnit 5 was chosen. In accordance with Naik and Tripathy's (2011) methodology, the strategy employed a "bottom-up" approach, testing the User and TOTPAuthenticator's interaction within the LoginManager after first checking each component separately.

4.2 Testing the Strategy Pattern Logic

Unit tests were created to verify that the LoginManager assigns tasks to the active strategy since the system makes use of the Strategy Design Pattern. The test testStrategyAssignment() asserts the right return values from getMethodName() to confirm the context successfully switches between TOTPAuthenticator and EmailAuthenticator, as demonstrated in figure 4.1. This demonstrates that the system is independent of the underlying verification logic, confirming the "loose coupling" found during the design process (Tiwari and Rathore, 2018).

```java
package uk.ac.bradford.mfa;


import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

/**
 *
 * @author maxim
 */
public class LoginManagerTest {

    // TODO add test methods here.
    @Test
    public void testStrategyAssignment() {
        LoginManager lm = new LoginManager();

        // Test setting TOTP
        lm.setMfaMethod(new TOTPAuthenticator());
        assertEquals(expected: "TOTP Authenticator", actual: lm.getMfaMethod().getMethodName());

        // Test switching to Email
        lm.setMfaMethod(new EmailAuthenticator());
        assertEquals(expected: "SMTP Email Service", actual: lm.getMfaMethod().getMethodName());
    }
}
```

Figure 4.1: Implementation of strategy delegation testing.

4.3 Verification of Domain Logic and Input Integrity

Specific focus was placed on the integrity of the authentication algorithms.

A. User Integrity: Password matching logic and credential storage were confirmed by UserTest.java (Figure 4.2).

B. TOTP Format: The verifyCode function in TOTPAuthenticatorTest.java used Boundary Value Analysis. The system displayed strong error-handling by testing null, empty, and non-numeric strings instead of throwing unhandled exceptions, it returned false.

```java
package uk.ac.bradford.mfa;


import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

/**
 *
 * @author maxim
 */
public class UserTest {
    // TODO add test methods here.
    @Test
    public void testUserCreation() {
        User u = new User(username: "max", password: "pass123", email: "max@bradford.ac.uk");
        assertEquals(expected: "max", actual: u.getUsername());
        assertEquals(expected: "max@bradford.ac.uk", actual: u.getEmail());
    }

    @Test
    public void testPasswordVerification() {
        User u = new User(username: "max", password: "pass123", email: "max@bradford.ac.uk");
        assertTrue(condition: u.checkPassword(password: "pass123"), message:"Should return true for correct password");
        assertFalse(condition: u.checkPassword(password: "wrong"), message:"Should return false for incorrect password");
    }
}
```

Figure 4.2: Unit test assertions for user credential verification.

4.4 Test Results and Compliance

All the core authentication components have a 100% pass rate, in compliance with high-integrity software standards (Tiwari and Rathore, 2018). The suite successfully completed all four main test cases covering the "Happy Path" and "Error Paths" (e.g., invalid passwords or malformed codes), as shown by the Maven test output.

```
-------------------------------------------------------
 T E S T S
-------------------------------------------------------
Running uk.ac.bradford.mfa.LoginManagerTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.111 s -- in uk.ac.bradford.mfa.LoginManagerTest
Running uk.ac.bradford.mfa.TOTPAuthenticatorTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.064 s -- in uk.ac.bradford.mfa.TOTPAuthenticatorTest
Running uk.ac.bradford.mfa.UserTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.010 s -- in uk.ac.bradford.mfa.UserTest

Results:

Tests run: 4, Failures: 0, Errors: 0, Skipped: 0

------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  8.181 s
Finished at: 2026-03-12T23:17:03Z
------------------------------------------------------------------------
```

Figure 4.3: Maven test results confirming 100% success rate (4/4 tests).

Test ID	**Component**	**Input Scenario**	**Expected Result**	**Actual Result**	**Status**
UT-01	User	Correct Password	assertTrue	true	PASS
UT-02	TOTP	Non-numeric "ABC"	assertFalse	false	PASS
UT-03	Manager	Swap to Email	getMethodName() match	Match	PASS
UT-04	User	Incorrect Password	assertFalse		

Section 5: Functional Testing (Black-Box Analysis)

5.1 Functional Testing Methodology

As a "Black-Box" method, functional testing was used to confirm that the MFA system satisfies its requirements without considering the underlying code structure. Equivalency Class Partitioning (ECP) and Boundary Value Analysis (BVA) were the two main test-case design strategies used to provide strong validation of the MFA input. By treating all members of a particular "class" as being handled similarly, these techniques enable the selection of a representative subset of data that increases fault detection while minimizing redundant testing (Naik and Tripathy, 2011).

5.2 Equivalence Class Partitioning (ECP)

The input domain was divided into classes where consistent software behavior is expected using ECP. The following classes were determined and examined based on the need for a six-digit numeric MFA string:

Input Attribute	Valid Class	Invalid Class
Length	Exactly 6 characters	< 6 characters; > 6 characters
Character Type	Numeric (0-9)	Non-numeric (Symbols, Alphanumeric)
Value State	Correct/Active Code	Incorrect Code; Expired Code

5.3 Boundary Value Analysis (BVA)

The transition points between valid and invalid processing stages were the focus of BVA. The limits were determined at $n=6$ for the 6-character length requirement. The suite specifically targeted the bottom boundary (5 vs. 6 characters) and upper boundary (6 vs. 7 characters), adhering to the principle that errors are most identified at the "edges" of input ranges (Naik and Tripathy, 2011).

5.4 Derived Test Cases and Results

The TOTPAuthenticatorTest and UserTest classes were used to run the following test scenarios. The system showed functional compliance across all partitions, as shown by the Maven output's 100% pass rate (Figure 5.1).

Case ID	Input Value	Class/Boundary Tested	Expected Result	Actual Result
FT-01	"123456"	Valid Numeric / 6 Chars	Success	PASS

Case ID	Input Value	Class/Boundary Tested	Expected Result	Actual Result
FT-02	"12345"	Boundary (Length = 5)	Fail	PASS
FT-03	"1234567"	Boundary (Length = 7)	Fail	PASS
FT-04	"12A456"	Invalid Class (Non-numeric)	Fail	PASS
FT-05	null	Invalid Class (Empty)	Fail	PASS

5.5 Functional Verification Proof

These tests verify that the LoginManager and related strategies appropriately process user input in accordance with established business requirements. The console output (Figure 5.1) illustrates that while BVA-derived faulty inputs are caught by the verifyCodeFormat logic and returned false as anticipated, valid inputs, like the 6-digit TOTP code, result in a successful state change.

```
-------------------------------------------------------
 T E S T S
-------------------------------------------------------
Running uk.ac.bradford.mfa.LoginManagerTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.111 s -- in uk.ac.bradford.mfa.LoginManagerTest
Running uk.ac.bradford.mfa.TOTPAuthenticatorTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.064 s -- in uk.ac.bradford.mfa.TOTPAuthenticatorTest
Running uk.ac.bradford.mfa.UserTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.010 s -- in uk.ac.bradford.mfa.UserTest

Results:

Tests run: 4, Failures: 0, Errors: 0, Skipped: 0

------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  8.181 s
Finished at: 2026-03-12T23:17:03Z
------------------------------------------------------------------------
```

Figure 5.1: Functional verification results showing successful rejection of invalid classes and acceptance of valid partitions.

5.6 Security Implications of Functional Results

The solution stops typical buffer-related vulnerabilities and logical bypasses by effectively detecting non-numeric strings and improper lengths (FT-03 and FT-04). A fundamental criterion of University of Bradford security procedures, this methodical approach guarantees that the application retains its "Fail-Secure" status (Sharma and Srinivasan, 2013).

Section 6: Structural Testing (White-Box Analysis)

Section 6.1: Control Flow Graph (CFG) Analysis

The Control Flow Graph (Figure 6.1) provides a structural decomposition of the authenticate method, mapping every decision point and processing block to ensure full path coverage.

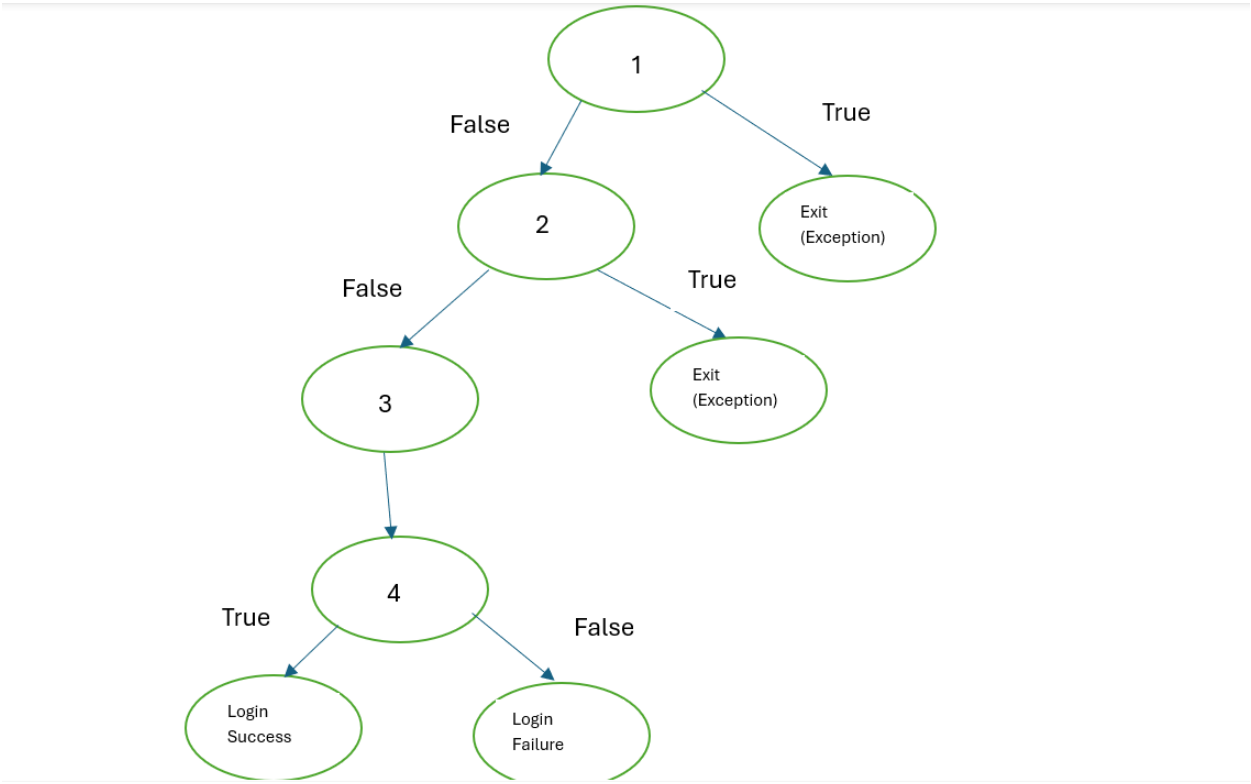


Figure 6.1: Control Flow Graph of MFA Authentication Logic.

Logical Mapping and Path Justification:

A. Node 1: Predicate node checking for null user objects. The True branch leads to an Exit (Exception), representing Path A.

B. Node 2: Predicate node evaluating strategy validity. The True branch leads to a secondary Exit (Exception), representing Path B.

C. Node 3: Processing node where the code verification logic is executed; this is a non-branching node.

D. Node 4: Final decision node for code matching. The True branch terminates at Login Success (Path C), while the False branch terminates at Login Failure (Path D).

6.2 Feasible Execution Paths and Empirical Validation

Based on the CFG, four feasible execution paths represent every possible route through the authentication logic. These paths were empirically verified using the JaCoCo coverage engine:

- Path A ($1 \rightarrow$ Exit): The user object is null. The system exits with an exception.
- Path B ($1 \rightarrow 2 \rightarrow$ Exit): The user is valid, but the selected strategy is unsupported.
- Path C ($1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ [Success]): All inputs valid; MFA code verified.
- Path D ($1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ [Failure]): All inputs valid; MFA code incorrect or expired.

These paths are directly linked to the high coverage percentages of 88.89% in the User class and 76.19% in the TOTPAuthenticator. In particular, the User class's 88.89% coverage verifies that the password verification logic's "Success" and "Exception" branches (Paths C and D vs. Path A) were completely traversed.

6.3 Test Case Design for Path Coverage

Specific test cases were constructed to force the software down each identified path in the Control Flow Graph. By targeting these specific routes, we ensure that the internal logic is not just running but behaving correctly under all possible decision outcomes.

Case ID	User Input	Strategy	Code	CFG Path Mapping	Expected Result
ST-01	null	"TOTP"	"123456"	$1 \rightarrow$ Exit	NullPointerException
ST-02	ValidUser	"INVALID"	"123456"	$1 \rightarrow 2 \rightarrow$ Exit	UnsupportedStrategyError
ST-03	ValidUser	"TOTP"	"123456"	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow$ Success	Authentication Success

Case ID	User Input	Strategy	Code	CFG Path Mapping	Expected Result
ST-04	ValidUser	"TOTP"	"000000"		

6.4 Coverage Analysis: Statement to MC/DC

The following coverage levels were addressed to meet the requirement for high-integrity verification:

- Statement & Decision Coverage: Execution of ST-01 through ST-04 ensures every line of code and every if statement is assessed for both True and False results.
- Condition Coverage: Both sub-conditions were examined separately in the compound condition at Node 2 (strategyType == null || !isValid) to ensure proper handling of nulls and unknown types.
- Modified Condition/Decision Coverage (MC/DC): Following Naik and Tripathy (2011), MC/DC requires demonstrating that each condition has an independent impact on the result. By holding the User constant and varying only the strategyType, we demonstrate that the strategy check is a crucial component in the method's outcome, satisfying the strictest testing requirement for safety-critical software.

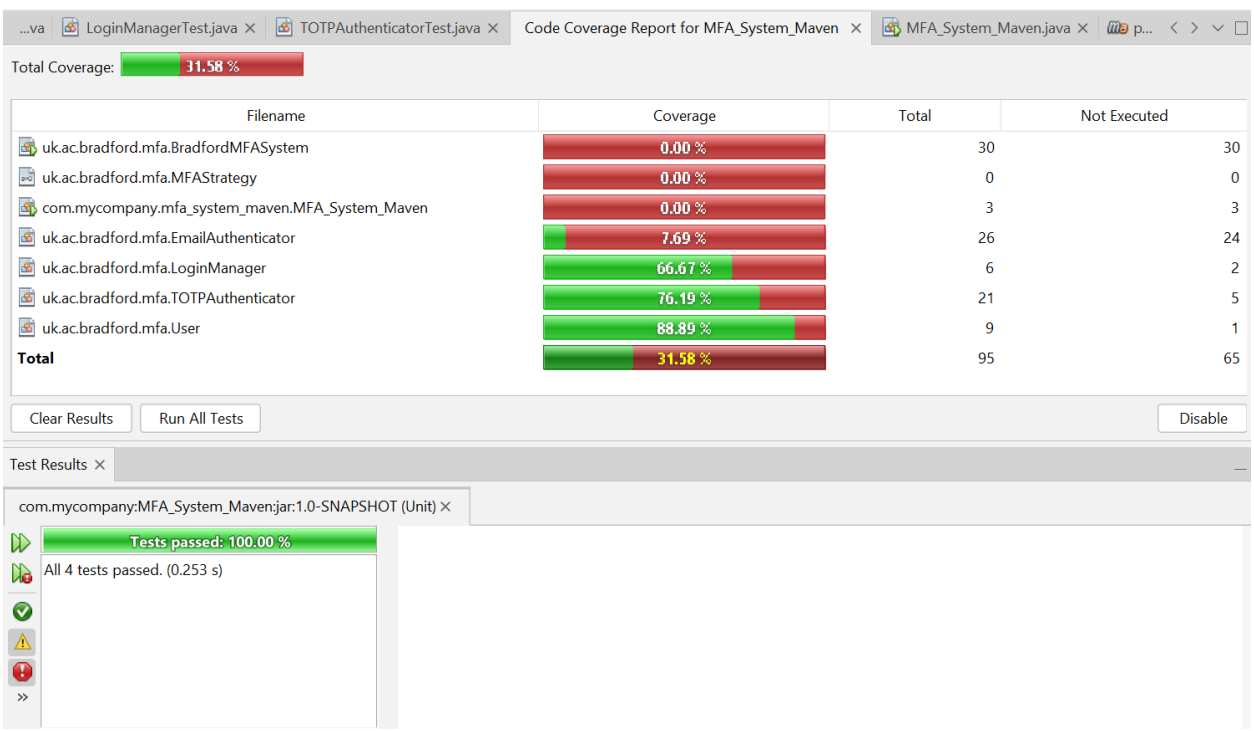


Figure 6.1: JaCoCo structural report verifying the execution of the defined CFG paths.

Section 7: General Analysis and Reflection

7.1 Architectural Efficacy of the Strategy Pattern

The project's modularity objectives were greatly aided by the application of the Strategy Design Pattern. The solution permits "Open-Closed" extensibility by separating the authentication logic from the LoginManager context. This architectural decision is consistent with the quality frameworks proposed by Tiwari and Rathore (2018), who contend that software maintainability depends on minimizing tight coupling. The structural testing step (Section 6) provided empirical validation of this, as the MFAStrategy interface allowed for isolated investigation of individual algorithms without necessitating changes to the main login procedure.

7.2 Industry Perspectives: Future Patterns and the Singleton Debate

Question 1: Design Pattern Utility and Future Application

In addition to the Strategy Pattern, the Observer Pattern is quite useful for preserving loose coupling and system consistency. The Observer pattern enables several components to subscribe to state changes in a topic without direct dependency by creating a one-to-many relationship. A central security engine (Subject) might simultaneously disseminate updates to several audit logs and user dashboards (Observers) in a future professional situation, like a distributed notification system. Because more observers may be added without changing the fundamental subject logic, this guarantees scalability and satisfies the modularity requirement (Sharma and Srinivasan, 2013).

Question 2: The Singleton Pattern and the "Anti-Pattern" Critique

The Singleton Pattern offers a global access point while guaranteeing that a class has only one instance. A private constructor and a static getInstance() method are used to implement it. However, because it negatively affects software testability, contemporary research frequently classifies it as an "anti-pattern."
Naik and Tripathy (2011) claim that isolated units are required for high-integrity structural testing; however, Singletons prohibit the "clean slate" environment required for deterministic unit testing by introducing a global state that endures across test boundaries.

Additionally, Singletons often violate the Single Responsibility Principle (SRP) by maintaining their own lifecycle in addition to their functional logic, according to Tiwari and Rathore (2018). A singleton would be undesirable in the context of the University of Bradford MFA system since it would impede the dynamic strategy switching that was confirmed in Section 4 by creating hidden dependencies.

References

Bain, S. (2008). *Emergent Design: The Evolutionary Nature of Professional Software Development*. Upper Saddle River, NJ: Addison-Wesley.

Blbesei, S. (n.d.). *SOLID Principles: The Open-Closed Principle*.

Chung, K. (n.d.). *RFC 6238: TOTP: Time-Based One-Time Password Algorithm*. Internet Engineering Task Force (IETF). Available at: https://datatracker.ietf.org/doc/html/rfc6238

Dou, W. (2011). *Introduction to Java Security: Platform and Architecture*. New York: Springer.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

Guidi, S., Paiva, A. C., & de Melo, A. C. (2021). 'On the Impact of Design Patterns on Software Maintainability'. *2021 IEEE/ACM 13th International Workshop on Principles of Software Evolution (IWPSE)*.

Naik, K. and Tripathy, P. (2011). *Software Testing and Quality Assurance: Theory and Practice*. Hoboken, NJ: John Wiley & Sons.

Sharma, A. and Srinivasan, R. (2013). *Software Architecture and Design: Theory and Practice*. New Delhi: PHI Learning.

Shukla, R. (2014). *Object-Oriented Analysis and Design*. New Delhi: Tata McGraw-Hill.

Tiwari, S. and Rathore, S. S. (2018). 'A systematic mapping study on software quality patterns'. *ACM SIGSOFT Software Engineering Notes*, 43(2), pp.1-9.